

universidad de león Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

Ingeniería del software seguro en la era de la IA generativa

Diseño y Programación Seguras

Confianza no calibrada

Francisco Javier Rodríguez Lera ▪ Camino Fernández Llamas



Instituto de Investigación e Innovación en Ingeniería

Escuela de Ingenierías

universidad de León

Objetivos de aprendizaje

Al terminar el capítulo, el estudiante será capaz de:

- **Distinguir** el riesgo técnico del *riesgo epistémico* del código generado.
- **Explicar** la trampa de la plausibilidad y el sesgo del caso positivo.
- **Calibrar** la confianza en tres dimensiones: especificación, verificación y reversibilidad.
- **Incorporar** el generador al modelo de amenazas.
- **Argumentar** la responsabilidad epistémica del aprobador.

Error técnico frente a riesgo epistémico

- El código incorrecto no es nuevo: la revisión, las pruebas y el modelado de amenazas existen porque la producción manual no garantiza corrección.
- Lo que distingue al código generado no es que sea incorrecto: es que es **plausiblemente incorrecto**.
- La distinción es *de naturaleza, no de grado*: el código plausible supera las comprobaciones que activan la desconfianza (compila, bien formateado, nombres coherentes, pasa los casos elegidos, encaja en los patrones).
- El defecto queda latente no porque nadie lo buscara, sino porque nadie supo que debía buscarlo.

Riesgo epistémico: no el riesgo de que el código sea malo, sino el de no saber cuánto confiar en él.

Hipótesis rectora del capítulo

- La plausibilidad superficial del código generado aumenta la probabilidad de confianza no calibrada respecto al código manual.
- Se etiqueta como **Hipótesis**: hay evidencia parcial (patrones documentados por el equipo de Claude Code [1]), pero no demostración sistemática y controlada.

Un riesgo técnico se mitiga con herramientas técnicas; un riesgo epistémico requiere, antes que herramientas, una teoría de qué se sabe y qué no se sabe del artefacto.

La trampa de la plausibilidad

- El cerebro infiere la calidad de señales de superficie. En código de un colega experto, esas señales *correlacionan* con comprensión real.
- Con un modelo de lenguaje, la correlación se rompe: optimiza la señal superficial directamente (aprende de código considerado bueno y reproduce sus propiedades de forma).
- El resultado *parece* correcto porque comparte propiedades de forma del código correcto, sin compartir sus propiedades de fondo.

El revisor activa un modo de revisión de *baja intensidad*: el código generado suprime las señales de alarma. **La plausibilidad no es una propiedad de seguridad.**

Formulación técnica y consecuencia

Formulación técnica. Un modelo aprende la distribución de probabilidad sobre secuencias de tokens en código de calidad y genera código probable bajo esa distribución. La probabilidad *no implica* corrección respecto a una especificación no observada.

Consecuencia práctica. Ante código bien formateado que compila y pasa los tests visibles, el revisor baja la intensidad; la revisión de alta intensidad se reserva a señales de alarma.

La trampa es, en parte, una trampa de la **competencia aparente**: el código generado puede ser más «listo» que el que el revisor habría producido.

El caso transversal: lo que el código plausible omite

El modelo produce código XLSX que importa la biblioteca correcta, itera de forma idiomática, usa nombres coherentes y genera un archivo válido. Pero *no*:

- Verifica que la ruta de salida esté dentro del directorio autorizado.
- Escapa los prefijos de fórmula en las celdas.
- Garantiza que no queda un archivo parcial en caso de error.
- Acota el tiempo y la memoria de la operación.

La omisión es *invisible*: el código que *debería* estar no produce ningún artefacto observable cuando no está.

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

¿Por qué no se nota lo que falta?

- Si el código funciona para los casos que se prueban, ¿qué señal avisa de que falta algo?

La confianza como variable continua

- No es binaria: hay un espectro entre «confío plenamente» y «no confío en absoluto».
- El problema es que el desarrollador toma una posición implícita —a menudo hacia la confianza alta— sin examinar la evidencia disponible.
- Se propone un marco de **tres dimensiones**; ninguna basta por sí sola.

Las tres dimensiones de calibración

Cobertura de especificación. ¿Qué fracción del comportamiento esperado está enunciada de forma verificable? La confianza no puede superar la cobertura de la especificación contra la que se verifica.

Independencia de verificación. ¿Quién verificó y qué supuestos comparte con el generador? Se mide por la *diversidad de supuestos*, no por el número de verificadores.

Reversibilidad. ¿Qué cuesta deshacer el cambio si falla? No reduce la probabilidad de fallo; reduce el coste de un fallo que ocurra.

Marco de calibración de confianza

Dimensión	Confianza alta justificada	Confianza alta injustificada
Cobertura de especificación	Contrato completo con invariantes verificables	Solo caso positivo; errores implícitos
Independencia de verificación	Revisor con contexto fresco, supuestos distintos	Comparte modelo, contexto o ejemplos
Reversibilidad	Cambio tras flag, rollback de un paso	Migración irreversible, API pública

Marco como **Hipótesis** operativa: su valor no es predictivo sino normativo; obliga a articular en qué dimensiones la confianza está respaldada.

Reversibilidad, recuperabilidad, resiliencia

Reversibilidad. Posibilidad de deshacer determinados efectos de una operación (retirar un cambio, desactivar tras un flag, deshacer una escritura). No implica restaurar el estado del sistema.

Recuperabilidad. Restablecer un estado de servicio aceptable tras una perturbación; puede requerir restauración, reconciliación o compensación.

Resiliencia. Capacidad más amplia de mantener, adaptar o recuperar funciones ante perturbaciones, incluso sin recuperación exacta.

Son distinciones operativas, no equivalencias normativas: un cambio puede ser reversible y dejar el sistema degradado. El tratamiento operativo corresponde al capítulo de metodología.

El sesgo del caso positivo y su circularidad

- Los modelos generan implementaciones que satisfacen el caso *descrito*. No es un defecto: es consecuencia de cómo aprende.
- **Circularidad cognitiva:** el desarrollador imagina casos relevantes, el modelo genera código que los maneja, el desarrollador prueba esos casos y el código los pasa.
- Los casos fuera del espacio compartido permanecen invisibles.

La circularidad no es exclusiva de la IA: también al escribir a mano se tiende a probar lo imaginado. La diferencia: con código generado el proceso de escritura *no ocurrió*.

Ruptura de la circularidad: pruebas negativas

```python
# Lo que el modelo genera de forma natural: casos positivos
def test_exportacion_normal():
    registros = [["nombre", "valor"], ["Alice", "42"]]
    exportar_xlsx(registros, DIRECTORIO / "resultado.xlsx")
    assert (DIRECTORIO / "resultado.xlsx").exists()
    # Pasa: lo imagino el modelo y tambien el desarrollador.

# Lo que exige salir del espacio compartido: casos negativos
def test_formula_en_primera_celda():
    registros = [["=HYPERLINK(\"http://malo\",\"clic\")", "valor"]]
    exportar_xlsx(registros, DIRECTORIO / "resultado.xlsx")
    libro = openpyxl.load_workbook(DIRECTORIO / "resultado.xlsx")
    celda = libro.active.cell(row=1, column=1).value
    assert not str(celda).startswith("="), f"Celda no escapada: {celda!r}"

def test_path_traversal_en_nombre():
    ruta = DIRECTORIO / "../secretos.xlsx"
    with pytest.raises(ValueError):
        exportar_xlsx([["dato"]], ruta)
```

Listing 1: La circularidad del caso positivo y su ruptura (ejemplo didáctico).

Las pruebas negativas como única salida sistemática

- Una prueba negativa no pregunta «¿hace lo que debe?» sino «¿rechaza lo que no debe aceptar?».
- Requiere especificar las invariantes y construir activamente entradas que intenten violarlas.
- Por definición, es *externa* al espacio de casos que el modelo considera naturales.
- Los casos positivos emergen del enunciado; los negativos requieren análisis externo de invariantes y vectores de amenaza.

Independencia de contexto como propiedad de diseño

- La revisión es inútil si el revisor comparte los supuestos del generador.
- En desarrollo manual, la revisión por pares da independencia casi automática: el revisor no estuvo en la sesión, no vio el razonamiento intermedio.
- Con IA, si el mismo desarrollador genera, revisa y testea con el mismo agente, las tres operaciones comparten contexto y distribución. **El número de participantes es uno, no tres.**

Tres formas de independencia

Independencia de modelo. Revisor con un modelo distinto; pueden compartir datos de entrenamiento, pero difieren en arquitectura, ajuste y distribución. Parcial pero no trivial.

Independencia de contexto. Revisor sin acceso al historial del generador; solo ve el artefacto y la especificación. *La más importante para seguridad.*

Independencia de enunciado. El enunciado del revisor es la especificación, no la descripción de lo que el generador hizo.

La independencia de contexto **no es una práctica de proceso**: es una *propiedad de diseño* del sistema de verificación.

DISCUSIÓN ¿Es independiente una revisión del mismo agente?

- Antes de considerar una revisión independiente: ¿qué supuestos comparte el revisor con el generador?

El generador en el modelo de amenazas

- Con desarrollo asistido, el modelo generador debe aparecer explícitamente en el modelo de amenazas del sistema.
- **No** como actor malicioso, sino como *fuente de código no verificado* con características de fallo propias y predecibles.
- Omitirlo es tan incompleto como omitir las dependencias de terceros o las entradas de usuario.

Características de fallo del generador

Característica	Manifestación	Control
Sesgo hacia el caso positivo	Precondiciones implícitas, errores ausentes	Especificación de invariantes; pruebas negativas
Plausibilidad sin corrección	Pasa revisión superficial con supuestos erróneos	Revisión independiente; pruebas de propiedad
Replicación de patrones inseguros	Patrones del corpus reproducidos con confianza	Revisión de seguridad; análisis estático
Ausencia de modelo del sistema	Supuestos erróneos sobre versiones, permisos, concurrencia	Especificar el entorno; pruebas de integración
Convergencia indefinida	Bucle generador-revisor sin terminación	Límite de rondas; revisión diferencial

Convergencia indefinida: un fallo de diseño

- El equipo de Claude Code documenta un agente autor y otro revisor interactuando más de cincuenta rondas sin convergencia, porque el revisor seguía detectando problemas en código preexistente no modificado [1].
- No es una curiosidad: es un fallo del sistema de verificación.
- Un proceso que no converge no es una propiedad de seguridad; es ruido que degrada la señal y agota la atención humana.
- Solución: límite de rondas, revisión diferencial, prohibición de repetir hallazgos ya publicados —instancia de la *economía de mecanismos* de Saltzer y Schroeder—.

Responsabilidad epistémica

- «Lo integras, lo posees» tiene lectura superficial (culpa) y profunda: es una **asignación de responsabilidad epistémica**.
- Quien aprueba y fusiona debe poder articular qué sabe y qué no sabe sobre el cambio.
- La responsabilidad no desaparece: se transforma. Antes incluía razonar sobre la implementación; ahora incluye razonar sobre la *verificación*.

«Si no puedes decir qué fallarían las comprobaciones, pide una revisión real en lugar de una aprobación rápida» [1] —instancia de la *aceptabilidad psicológica* de Saltzer y Schroeder—.

Tres preguntas de una aprobación válida

1. ¿Qué invariantes de seguridad debe preservar este código?
2. ¿Qué verificación cubre cada invariante?
3. ¿Qué queda sin cubrir y qué riesgo implica?

La aprobación de un agente **nunca** cuenta como aprobación de una persona, ni en caminos de bajo riesgo: un agente aprobando a otro con supuestos compartidos no añade independencia epistémica. Si alguna pregunta no tiene respuesta, la aprobación es injustificada.

«El agente dijo que era correcto»

- ¿Es eso evidencia de corrección o ausencia de evidencia negativa?

Qué abarata y qué encarece la IA

Dimensión	Lo que abarata la IA	Lo que exige o encarece
Producción de artefactos	Generar código plausible en volumen	Saber cuánto confiar en cada artefacto
Cobertura de especificación	Borradores de contrato a partir del código	Validar que captura las invariantes
Verificación	Escalar la revisión automática	Mantener independencia de contexto
Pruebas	Producir casos positivos	Diseñar casos negativos
Responsabilidad	Argumentos de corrección plausibles	Articular qué sabe y qué no el aprobador
Gestión del riesgo	Rollback y flags	Calibrar la confianza según reversibilidad

La IA abarata la producción de artefactos sin reducir proporcionalmente el coste epistémico de saber cuánto valen. No es un defecto que futuras versiones resolverán: es *estructural*.

Síntesis del capítulo

- El riesgo específico no es que el código sea incorrecto, sino que la señal que activa la revisión cuidadosa *está ausente*: es plausiblemente incorrecto.
- Tres compromisos que no se delegan en la herramienta:
  1. **Especificar antes de generar** (no inferir el contrato del código generado).
  2. **Verificar con independencia de contexto** (modelo, contexto o enunciado distintos).
  3. **Calibrar la confianza explícitamente** (cobertura, independencia, reversibilidad).
- No son nuevos: distinguen el desarrollo disciplinado del descuidado. La IA aumenta la *urgencia* de hacerlos explícitos.

Actividad

1. **Diagnóstico de plausibilidad:** pida una función de exportación sin especificar invariantes; identifique tres propiedades no garantizadas que un revisor asumiría.
2. **Calibración:** aplique el marco de tres dimensiones a un cambio propio o al caso transversal. ¿Hay confianza alta sin respaldo?
3. **Ruptura de circularidad:** diseñe cinco pruebas negativas fuera del espacio del enunciado.
4. **Independencia:** proponga un protocolo de revisión que maximice la independencia de contexto con un solo modelo.
5. **Responsabilidad:** distinga ausencia de evidencia negativa de evidencia positiva.

¿Reduce la IA el riesgo epistémico?

- ¿Automatizar el desarrollo reduce el riesgo epistémico o lo desplaza?

Referencias I


Addy Osmani, Mark W. Lunt, and Qing Wang.
How the claude code team does code review.
claude.dev Blog, 15 de octubre de 2026, 2026.
Testimonio del equipo Claude Code; no es evidencia experimental independiente.